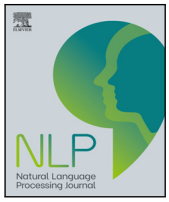
Natural Language Processing Journal 7 (2024) 100076

Contents lists available at ScienceDirect

Natural Language Processing Journal

journal homepage: www.elsevier.com/locate/nlp



Challenges and Opportunities of Using Transformer-Based Multi-Task Learning in NLP Through ML Lifecycle: A Position Paper

Lovre Torbarina [*,1], Tin Ferkovic[1], Lukasz Roguski[2], Velimir Mihelcic, Bruno Sarlija, Zeljko Kraljevic

doXray B.V., Borculoseweg 41, Neede, 7161 GR, Gelderland, Netherlands

ARTICLE INFO

Keywords:
Natural language processing
Multi-task learning
Machine learning lifecycle

ABSTRACT

The increasing adoption of natural language processing (NLP) models across industries has led to practitioners' need for machine learning (ML) systems to handle these models efficiently, from training to serving them in production. However, training, deploying, and updating multiple models can be complex, costly, and time-consuming, mainly when using transformer-based pre-trained language models. Multi-Task Learning (MTL) has emerged as a promising approach to improve efficiency and performance through joint training, rather than training separate models. Motivated by this, we present an overview of MTL approaches in NLP, followed by an in-depth discussion of our position on opportunities they introduce to a set of challenges across various ML lifecycle phases including data engineering, model development, deployment, and monitoring. Our position emphasizes the role of transformer-based MTL approaches in streamlining these lifecycle phases, and we assert that our systematic analysis demonstrates how transformer-based MTL in NLP effectively integrates into ML lifecycle phases. Furthermore, we hypothesize that developing a model that combines MTL for periodic re-training, and continual learning for continual updates and new capabilities integration could be practical, although its viability and effectiveness still demand a substantial empirical investigation.

1. Introduction

In recent years, advancements in natural language processing (NLP) have revolutionized the way we deal with complex language problems. Consequently, those advances have been significantly impacting the global industry, driving growth in organizations that incorporate Artificial Intelligence (AI) technologies as part of their core business.

To illustrate, McKinsey's state of AI report for 2022 reported an increase of 3.8 times since 2017 in AI capabilities that organizations have embedded within at least one function or business unit, where natural language understanding (NLU) took third place among reported capabilities, just behind computer vision (Chui et al., 2022). Additionally, McKinsey's State of AI 2023 report highlighted that more than two-thirds of respondents anticipate their organizations will increase AI investment over the next three years (Chui et al., 2023). Furthermore, Fortune Business Insights reported that the global NLP market was valued at USD 24.10 billion in 2023 (Fortune Business Insights, 2024). The report projected that the market will grow from USD 29.71 billion in 2024 to USD 158.04 billion by 2032. As a result, every NLP practitioner offering models through an API or using them internally, alone, or together with other AI capabilities, should have a machine learning (ML) system to efficiently manage these models. This involves having well-established processes from training and verifying those models to deploying them in production for end-users while continuously monitoring that those models remain up-to-date with the most recent knowledge they are being trained on.

This trend of widespread adoption of ML models by various practitioners throughout industries, and the resulting need for ML systems to manage them efficiently, was tackled in a survey of ML systems conducted by Paleyes et al. (2022). The survey analyzed publications and blog posts reported by different practitioners, providing insights into phases of an ML lifecycle and challenges that commonly arise during those phases. The ML lifecycle refers to the phases and processes involved in designing, developing, and deploying an ML system. It encompasses the entire process, from defining the problem and collecting data to deploying models and monitoring their performance. Model learning and model deployment are two important phases in the ML lifecycle, among others (Ashmore et al., 2021; Paleyes et al., 2022). In many cases, to support practitioners' needs, the model learning phase should be equipped to handle the training and updating of a large

* Corresponding author.
E-mail addresses: lovre.torbarina@doxray.com (L. Torbarina), tin.ferkovic@doxray.com (T. Ferkovic), velimir.mihelcic@doxray.com (V. Mihelcic), bruno.sarlija@doxray.com (B. Sarlija), zeljko.kraljevic@doxray.com (Z. Kraljevic).

[1] Equal contribution.

[2] Work done while part of doXray B.V.

https://doi.org/10.1016/j.nlp.2024.100076
Received 3 November 2023; Received in revised form 2 April 2024; Accepted 18 April 2024

number of models, while the model deployment phase should provide an easy and efficient way to integrate and serve those models to run in production, that is, running as part of usual business operations.

At the same time, it is a common practice in NLP production systems to utilize pre-trained language models based on transformers (Vaswani et al., 2017) by fine-tuning them for specific downstream tasks. While effective, the language models have a large number of parameters that require significant computational resources to fine-tune. Although fine-tuning pre-trained models can be more data-efficient than training a model from scratch, the expertise of annotators or domain experts may still be required to label a large number of examples, particularly if there is a significant difference between the downstream task and pre-training objectives (Wang et al., 2020). Therefore, it is a costly and time-consuming procedure, especially if there is a need to train and serve multiple models in production. To address the challenge of training multiple models, researchers have been exploring Multi-Task Learning (MTL) as a solution (Ruder, 2017). MTL trains a single model to learn multiple tasks simultaneously while sharing part of the model parameters between them (Caruana, 1997), making the process memory-efficient and, in some cases, computationally more efficient than training multiple models. We posit that using a single model for multiple downstream tasks in a production system has the potential to simplify the integration of ML models with ML systems and reduce economic costs. This potential arises from the modular nature of MTL architectures that promote code, data, and model sharing, reuse, easier collaboration, and maintenance. Therefore, we advocate that MTL approaches offer a promising solution to mitigate some of the difficulties associated with managing multiple models in ML production systems.

In this position paper, we begin by providing an overview of transformer-based MTL approaches in NLP (see Section 3). Following this overview, we then highlight the opportunities of using MTL approaches across multiple stages of the ML lifecycle, emphasizing our perspective on the challenges related to data engineering, model development, deployment, and monitoring (see Section 4). Our discussion is particularly centered on transformer-based architectures. To the best of our knowledge, this position paper is the initial attempt to systematically discuss the benefits of using MTL approaches across multiple ML lifecycle phases (see Section 2). Additionally, we suggest exploring the relationship between MTL and Continual Learning (CL). MTL performs well in periodic re-training, while CL offers advantages for continuous updates in response to distribution shifts and the integration of new capabilities into production models. We posit that a model combining both MTL and CL could be practical for production systems. However, the practical implementation of such a model and its efficacy are subjects for future research.

The rest of the paper is organized as follows. In Section 2, we briefly review related surveys and highlight the gaps our arguments aim to address. In Section 3, we provide an overview of transformer-based MTL approaches, laying the foundations necessary for understanding our subsequent arguments. In Section 4, we systematically advocate for the benefits of using MTL through specific ML lifecycle phases. And finally, in Section 5, we summarize our arguments to offer a conclusion to our work.

2. Related surveys

In this section, we give an overview of related work on MTL, ML systems, and CL, and point out what has not been discussed so far in the literature regarding the connection of MTL to both ML systems and CL.

2.1. Multi-task learning

The idea of MTL was explored in many research studies. In this section, we provide an overview of related MTL surveys, address various aspects of MTL, and list them along with their corresponding surveys in Table 1. In the rest of the section, we just mention related surveys and go over individual MTL aspects (shown in **bold**).

Application domains. Many application domains were studied in previous work, ranging from surveys covering multiple domains (Ruder, 2017; Zhang and Yang, 2017, 2018; Thung and Wee, 2018; Vafaeikia et al., 2020; Crawshaw, 2020; Upadhyay et al., 2021; Abhadiomhen et al., 2022), to those dedicated to a specific domain, such as computer vision (Vandenhende et al., 2021) or natural language processing (Zhou, 2019; Worsham and Kalita, 2020; Chen et al., 2021; Samant et al., 2022; Zhang et al., 2023b). Both traditional ML and deep learning **computational models** were studied. The traditional ML was discussed primarily in older studies, while deep learning was represented in all but one study. Furthermore, most prior works provided an overview of the **benchmarks** for the specific domain McCann et al. (2018), Wang et al. (2019), and compared models on them.

Learning types. Two learning types were mainly discussed. First, joint learning was used in an MTL setting where all the tasks are equally important (Kendall et al., 2018; Liu et al., 2019a). Here, the goal is to achieve an on-par performance compared to their single-task learning (STL) counterparts. Second, auxiliary learning was used when there was a single main task or a set of them, while auxiliary tasks are only used to improve the performance of the main tasks (González-Garduño and Søgaard, 2018; Wang et al., 2022). Although the difference between the two learning types can be made, sometimes auxiliary learning was not differentiated from joint learning.

Architectures. MTL model architectures were among the most discussed aspects of MTL and one of the first aspects tackled in previous work. Distinguishing between hard and soft parameter sharing (Ruder, 2017) is the most used architecture taxonomy. In recent surveys, the method of sharing parameters between tasks has improved, leading to the refinement of taxonomies to categorize architectures more precisely (Crawshaw, 2020; Chen et al., 2021). Next, some MTL architectures were categorized as learning-to-share approaches (Ruder et al., 2017). Those works argue that it is better to learn parameter sharing in architectures for MTL rather than hand-design where sharing happens, as it offers a more adaptive solution to accommodate task similarities at different parts of the network. Additionally, some MTL architectures were categorized as universal models that can handle multiple modalities, domains, and tasks with a single model (Kaiser et al., 2017; Pramanik et al., 2019).

Optimization. Optimization techniques for MTL architectures were also discussed in detail. To start with, the most common approach to mitigating MTL challenges is loss weighting. Computing weights of task-specific losses is crucial, as it helps optimize the loss function and considers the relative importance of each task. There are various approaches to computing the loss weights dynamically, including weighting by uncertainty (Kendall et al., 2018), learning speed (Liu et al., 2019b; Zheng et al., 2019), or performance (Guo et al., 2018; Jean et al., 2019), among others. Next, and closely related to weighting task losses, is a task scheduling problem that involves choosing tasks to train on at each step. Many techniques were used, from simple ones that employ uniform or proportional task sampling, to the more complicated ones, such as annealed sampling (Stickland and Murray, 2019) or approaches based on active learning (Pilault et al., 2020). Regularization approaches were also analyzed. Methods include (1) minimizing the L2 norm between the parameters of soft-parameter sharing models (Yang and Hospedales, 2016), (2) placing prior distributions on the network parameters (Long et al., 2017), (3) introducing an auto-encoder term to the objective function (Lee et al., 2018), (4) MTL variant of dropout (Pascal et al., 2021), and others. To continue, gradient modulation techniques were employed to mitigate the problem of negative transfer by manipulating gradients of contradictory tasks, either through adversarial training (Sinha et al., 2018) or by replacing the gradient with its modified version (Lopez-Paz and Ranzato, 2017). Another approach for optimizing MTL models is by applying knowledge

Table 1
Discussed aspects per MTL survey. Aspects are indicated in **bold**.

Year	MTL survey													
2017	1 - Ruder (2017) 2 - Zhang and Yang (2017)													
2018	3 - Zhang and Yang (2018) 4 - Thung and Wee (2018)													
2019	5 - Zhou (2019)													
2020	6 - Vafaeikia et al. (2020) 7 - Worsham and Kalita (2020) 8 - Crawshaw (2020)													
2021	9 - Vandenhende et al. (2021) 10 - Chen et al. (2021) 11 - Upadhyay et al. (2021)													
2022	12 - Samant et al. (2022) 13 - Abhadiomhen et al. (2022)													
2023	14 - Zhang et al. (2023b)													
Aspect \Survey	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Computational Model														
Traditional ML	✓	✓	✓	✓									✓	
Deep Learning	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓		✓
Learning Type														
Joint Learning	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Auxiliary Learning	✓	✓	✓	✓	✓	✓	✓			✓		✓		✓
Architectures														
Taxonomy	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Learning to Share	✓			✓	✓	✓		✓	✓	✓				✓
Universal Models					✓			✓	✓	✓				✓
Optimization														
Loss Weighting	✓	✓	✓		✓	✓	✓	✓	✓	✓		✓		
Regularization	✓	✓	✓	✓	✓			✓		✓			✓	
Task Scheduling					✓	✓	✓	✓	✓	✓				
Gradient Modulation	✓					✓		✓	✓	✓				
Knowledge Distillation							✓	✓		✓		✓		
Multi-Objective Optimization								✓	✓	✓				
Task Relationship Learning														
Task Grouping	✓	✓	✓	✓			✓	✓	✓				✓	✓
Relationships Transfer	✓			✓			✓	✓	✓					
Task Embeddings							✓	✓						
Supervision Level														
Supervised Learning	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Semi-supervised Learning		✓	✓							✓		✓		✓
Self-supervised Learning		✓	✓				✓			✓		✓		✓
Connection to Learning Paradigm														
Reinforcement Learning		✓	✓		✓	✓		✓						✓
Transfer Learning		✓	✓	✓										
Domain Adaptation	✓						✓				✓			
Meta-Learning								✓			✓			✓
Active Learning		✓	✓											
Online Learning	✓	✓	✓											
Continual Learning														
Benchmarks														
Benchmark Overview			✓				✓	✓	✓	✓		✓	✓	
Model Comparison			✓				✓	✓	✓	✓		✓	✓	
Application Domain														
Natural Language Processing	✓	✓	✓	✓	✓		✓	✓		✓	✓	✓		✓
Computer Vision	✓	✓	✓	✓	✓	✓		✓	✓		✓			
Healthcare		✓	✓	✓										
Bioinformatics	✓	✓	✓	✓							✓			
Other	✓	✓	✓					✓					✓	

distillation (Clark et al., 2019a). Finally, multi-objective optimization has been applied to the MTL setting to obtain a set of Pareto optimal solutions on the Pareto frontier, providing greater flexibility in balancing trade-offs between tasks (Lin et al., 2019).

Task relationship learning. The approach in MTL that focuses on learning the explicit representation of tasks or relationships between them is task relationship learning. This approach consists of three main categories of methods. First, task grouping aims to divide a set of tasks into groups in order to maximize knowledge sharing during joint training (Standley et al., 2020). Second, transfer relationship learning involves methods that determine when transferring knowledge from one task to another will be beneficial for joint learning (Zamir et al., 2018; Guo et al., 2019). Finally, task embedding methods aim to learn an embedding space for the tasks themselves (Vu et al., 2020).

Supervision level. Most studies focused on supervised approaches. Some studies analyzed semi-supervised approaches that incorporate self-supervised objectives, such as MLM. However, self-supervised approaches are mainly discussed in the context of pre-training.

Connections to other learning paradigms. Most studies had made connections to other learning paradigms, including reinforcement learning, transfer learning with a special emphasis on domain adaptation, meta-learning, and active and online learning. However, only Ruder (2017), Zhang and Yang (2017, 2018) explored the relationship between MTL and online learning in the context of traditional

ML. To the best of our knowledge, there had been no prior work systematically investigating connections between MTL and CL. We believe that a connection between MTL and CL represents a promising research direction, as we will motivate the importance of this connection in Section 4.

2.2. ML lifecycle and ML systems

The unparalleled growth of advancements in ML methods in recent years, with applications in NLP, computer vision, and others, has increased the complexity of building ML systems that need to address the requirements of an ML lifecycle. Previous works reviewed the challenges of such systems, often defining ML lifecycle phases such as data management, model learning, and model deployment, to study systematically the workings of ML systems and identify challenges within and across phases (Vartak and Madden, 2018; Ashmore et al., 2021; Paleyes et al., 2022; Huyen, 2022; Lavin et al., 2022; Cabrera et al., 2023; Nahar et al., 2023).

For example, Vartak and Madden (2018) defined ML lifecycle phases and analyzed model management challenges, while Ashmore et al. (2021) discussed assurance of ML for each phase. To streamline high-quality development, akin to following well-defined processes and testing standards seen in engineering systems, Lavin et al. (2022) proposed the Machine Learning Technology Readiness Levels (MLTRL) framework as a method for the development and deployment of robust, reliable, and responsible ML and data systems. The framework outlines 10 readiness levels, covering the phases from research and development to productization and deployment. Different from traditional ML workflows that follow a linear progression from research to deployment (Google, 2024), the MLTRL framework emphasizes the necessity for ML development to repeatedly cycle through the phases to achieve maturity and robustness. Paleyes et al. (2022) reviewed practitioners' challenges at each phase of an ML model deployment workflow in a broader scope than previous surveys. Cabrera et al. (2023) conducted an in-depth survey of real-world ML-powered systems through the lens of Data-Oriented Architecture (DOA), in which DOA proposes three core principles for building data-oriented software: treating data as a first-class citizen, prioritizing decentralization, and ensuring openness. The study explored how these DOA principles could assist practitioners in addressing the challenges associated with deploying ML models within larger systems. To do so, Cabrera et al. (2023) mapped the challenges identified in ML workflow deployment from the work of Paleyes et al. (2022) to the principles of DOA, illustrating which principles support addressing the respective challenges. Nahar et al. (2023) compiled and categorized more than 500 software engineering challenges encountered in the development of ML products, synthesizing information from 50 papers that collectively surveyed over 4,758 practitioners. For each category, the authors provided a high-level overview of the identified issues. Furthermore, Hu and Chechik (2023) introduced an approach for enhancing the safety and trustworthiness of systems incorporating ML components by proposing a feature-based analysis of the ML Development Lifecycle (MLDL). In this context, features are high-level abstractions of desired functionalities, model behavior, and data, studied across all MLDL stages. To achieve this, Hu and Chechik (2023) established a taxonomy of existing feature definitions used at various MLDL stages to enable the mapping of features across different stages with the goal of identifying gaps and future research directions, and highlighting potential collaboration areas between Software Engineering and other MLDL experts. In the rest of this section, we provide a few examples of challenges that occur during typical phases of the ML lifecycle.

There are many challenges that occur during different phases of the ML lifecycle. For example, data management is typically the early phase of the ML lifecycle with associated challenges such as data collection and preprocessing (Polyzotis et al., 2018; Sambasivan et al., 2021; Whang et al., 2023). Subsequently, model learning and verification phases take place, presenting challenges such as selecting (Ding et al., 2018) and training (Sun, 2020) a model, and determining the most effective method for verifying it (Bernardi et al., 2019; Schröder and Schulz, 2022), respectively. Then, a model deployment phase takes place with challenges such as model integration (Sculley et al., 2015; Renggli et al., 2019) into production. Finally, the monitoring phase with challenges such as continuous model performance monitoring (Schröder and Schulz, 2022) and updating a model over time (Ditzler et al., 2015; Abdelkader, 2020).

Some challenges can impact several phases of the ML lifecycle, such as collaboration among diverse teams and roles, including software and data engineers, data scientists, and other stakeholders (Takeuchi and Yamamoto, 2020; Nahar et al., 2022; Pei et al., 2022; Yang et al., 2022). Furthermore, there are challenges of bias, fairness, and accountability in ethics (Mehrabi et al., 2021; Kim and Doshi-Velez, 2021), various regulations set by law (Marchant, 2011; Politou et al., 2018) and adversarial attacks in security (Ren et al., 2020; Rosenberg et al., 2021), among others.

Previous works discussed MTL aspects to varying degrees in specific phases, either in a straightforward manner or indirectly. However, a systematic discussion of the potential benefits of using MTL approaches to alleviate challenges across different phases of the ML lifecycle has not been conducted.

2.3. Continual learning

CL incrementally learns a sequence of tasks, with a goal to progressively expand acquired knowledge and utilize it for subsequent learning (Chen and Liu, 2018). CL aims to overcome catastrophic forgetting (CF) and facilitate knowledge transfer (KT) across tasks, where CF is the degradation of performance on previous tasks when learning new ones, and KT is the ability to apply knowledge from past tasks to new tasks (Ke and Liu, 2022). Previous reviews on CL (Hsu et al., 2018; De Lange et al., 2021) categorized CL settings based on the marginal output and input distributions $P(Y^{(t)})$ and $P(X^{(t)})$ of a task t, with $P(X^{(t)}) \neq P(X^{(t+1)})$. First, class incremental learning is characterized by an expanding output space with observed class labels such that $Y^{(t)} \subset Y^{(t+1)}$ and $P(Y^{(t)}) \neq P(Y^{(t+1)})$. Second, task incremental learning (TIL), requires a task label t to identify the separate output nodes $Y^{(t)}$ for the current task t, where $Y^{(t)} \neq Y^{(t+1)}$. Lastly, incremental domain learning defines tasks with equal class labels and probability distributions, $Y^{(t)} = Y^{(t+1)}$, and $P(Y^{(t)}) = P(Y^{(t+1)})$.

CL approaches were also categorized into three main categories based on how task-specific information is stored and utilized during the incremental learning process. First, replay methods store samples in raw format or generate pseudo-samples with a generative model, replaying them while learning a new task to mitigate forgetting and prevent previous task interference. Second, regularization-based methods avoid storing raw inputs and reduce memory requirements by introducing an extra regularization term in the loss function to consolidate prior knowledge while learning new data. Third, parameter isolation methods allocate different model parameters to each task, either by adding new task-specific branches or masking out previous task parts, to prevent forgetting and maintain task-specific knowledge. We refer readers to Ke and Liu (2022) for more refined CL taxonomy and details in NLP.

In CL, MTL was usually used as an upper-bound baseline which can use all of the data from all of the tasks simultaneously (De Lange et al., 2021; Ke and Liu, 2022). Since CL and MTL work in different learning settings, few works tried to connect the two paradigms. Sun et al. (2020) presented a continual pre-training framework named ERNIE 2.0 which incrementally builds pre-training tasks and then learns pre-trained models on these constructed tasks via continual multi-task learning. Subsequently, ERNIE 2.0 was tested against CL and MTL pre-training approaches to evaluate the impact on abstractive text summarization task but performed similarly to other approaches (Kirstein et al., 2022). In Section 4.4, we motivate further research on combining CL and MTL approaches, as pre-training approaches are not sufficient for handling distribution shifts and adjusting models for new business requirements in real-world scenarios.

3. Multi-task learning approaches

3.1. Taxonomy

There were various MTL taxonomies covered in the surveys presented in Section 2.1. Ruder (2017) distinguishes between a *hard* and *soft* parameter sharing, which proved to be an influential taxonomy, since it was used in later works as well. Zhang and Yang (2018) defines three categories of multi-task supervised learning — *feature-*, *parameter-*, and *instance-based*. Vandenhende et al. (2021) distinguishes between *encoder-focused* and *decoder-focused* architectures. Chen et al. (2021) discusses *parallel, hierarchical, modular*, and *generative adversarial architectures*. We categorized transformer-based MTL approaches into 4 main categories based on differences in architectures: (1) Fully-Shared Encoder, (2) Adapters, (3) Hypernetworks, and (4) Prompting (Fig. 1).[3]

We will now argue on the choice of this taxonomy in comparison to the previous ones stated above. To start with, the taxonomy of Ruder (2017) focuses on a broad extent of sharing and provides a higher level of understanding, whilst ours provides a more nuanced understanding of how MTL can be implemented. In addition, Ruder (2017)'s soft parameter sharing generally requires a separate model per task, along with regularization of parameters between the models. During training, this leads to an increased number of parameters due to separate models and/or longer training times due to regularization. For that reason, we did not cover any soft parameter sharing approaches as we believe these are not straightforward to include in an ML lifecycle. Next, the taxonomy of Zhang and Yang (2018) offers a high-level view, focusing on areas where sharing occurs, whilst ours details specific architectural choices and emphasizes the mechanism of sharing (e.g., encoder, adapter, hypernetwork). The majority of our methods would fall into the Zhang and Yang (2018)'s parameter-based learning category. Additionally, our taxonomy puts focus on the effective and efficient approaches (Liu et al., 2019a; Hu et al., 2021; Pilault et al., 2020), while Zhang and Yang (2018) puts focus on completeness, covering much older works that lost relevance in a fast-paced AI's ecosystem. To continue, Vandenhende et al. (2021) again offers a broader, more general overview. Our fully-shared encoder approach would fall into their encoder-based architecture category, while all the other approaches can be applied to encoder, decoder, and encoder–decoder architectures. Finally, the taxonomy of Chen et al. (2021) looks more generally into how tasks are arranged and interact within the broader architecture. Fully-shared encoder would fall into a parallel architecture category, adapters and hypernetworks into a modular one, while prompting is not considered in their taxonomy. Overall, other works focus on broadness and completeness, while we focus on specific state-of-the-art methods, with an emphasis on practical applications. In Section 4, we refer to specific methods that we introduce here.

3.2. MTL approaches overview

3.2.1. Fully-shared encoder

A simple and intuitive approach to MTL is to have a clear division between shared and task-specific parameters. In such an approach, there is a shared transformer-based encoder in lower layers, while the top layers consist of different, task-specific layers (heads). One such approach, MT-DNN (Liu et al., 2019a), batches all the GLUE tasks (Wang et al., 2018) together and updates the model accordingly. The shared encoder is updated for all the instances, while the task-specific heads are updated only for the instances of a task that they are specific for. There are several downsides to this MT-DNN approach. First, task interference is not taken into account and the authors simply hope that the tasks would interact well, although some of the tasks are in different domains. Next, proportional random sampling is used, which could lead to underfitting on low-resource datasets. Finally, the loss is calculated in three different ways (for classification, regression, and ranking), and as a result, it has different scales. Nevertheless, all the loss functions are weighted equally (Zhou, 2019). Notwithstanding these observations, MT-DNN outperforms fine-tuning a different BERT (Devlin et al., 2019) model on most of the tasks. Additionally, Liu et al. (2019a) tried fine-tuning this multi-task model further on each task separately after training jointly on all the tasks, producing N models for N tasks. That again gave an improvement and a state-of-the-art performance at the time. However, an obvious downside is having a different model for each task.

Another shared-encoder approach is pre-finetuning. Muppet (Aghajanyan et al., 2021) shared an encoder in an MTL on 46 diverse datasets. Heterogeneous batches have proved beneficial in handling noisy gradients from different tasks. Furthermore, to have stable training, the data-point loss was divided by $\log(n)$, where n denotes the cardinality of the label set for the associated task. The authors maintained a natural distribution of datasets as other approaches led to degraded performance. The authors also found a threshold of around 15 tasks, below which downstream fine-tuning performance is degraded, and above which the performance improves linearly in the number of pre-finetuning tasks. A similar approach with an added decoder, EXT5 (Aribandi et al., 2021), extended the mixture to 107 supervised tasks, formatted them for encoder–decoder architectures, and performed pre-finetuning along with unsupervised T5's C4 span denoising (Raffel et al., 2020). Their mixture of tasks also included NLP applications such as reading comprehension, closed-book question answering, commonsense reasoning, dialogue, and summarization, among others. This showed that encoder–decoder models like T5 are capable of solving a wider range of NLP applications than encoder ones. However, task-specific models still achieve a better performance than general ones (Chung et al., 2022).

3.2.2. Adapters

Before being used in NLP, adapter residual modules were first introduced for the visual domain Rebuffi et al. (2017). Adapters are small, task-specific modules that are typically inserted within network layers, but can also be injected in parallel to them. In this paper, the network is always a transformer-based architecture. Compared to transformers' sizes, they add a negligible number of parameters per task. The parameters of the original network remain frozen unless stated otherwise, resulting in a high degree of parameter sharing and a small number of trainable parameters. Consequently, adapters for new tasks can be easily added without retraining the transformer or other adapters. They learn task-specific layer-wise representation, are small, scalable, and shareable, and have modular representations and a non-interfering composition of information (Pfeiffer et al., 2020b). Since the respective adapters were trained separately, the necessity of sampling heuristics due to skewed data set sizes no longer arises (Pfeiffer et al., 2020b).

AdapterHub. AdapterHub (Pfeiffer et al., 2020b) is a framework that allows dynamic usage of pre-trained adapters for different tasks and languages.[4] The framework is built on top of the HuggingFace Transformers library and enables quick and easy adaptation of state-of-the-art pre-trained models. It allows for efficient parameter sharing between tasks by training many task- and language-specific adapters, which can be exchanged and combined post-hoc. One can choose to stack the adapters on top of each other, combine them with attention (Pfeiffer et al., 2020a), or replace them dynamically. Downloading, sharing, and training adapters require minimal changes in the training scripts. More recently, a `peft` library (Mangrulkar et al., 2022) was

[3] In Appendix A.1, we give a brief overview of prompt engineering approaches as a fourth category. However, we do not include it in the main paper due to its need for a large number of parameters to perform well, making it inaccessible to most practitioners.

[4] https://adapterhub.ml

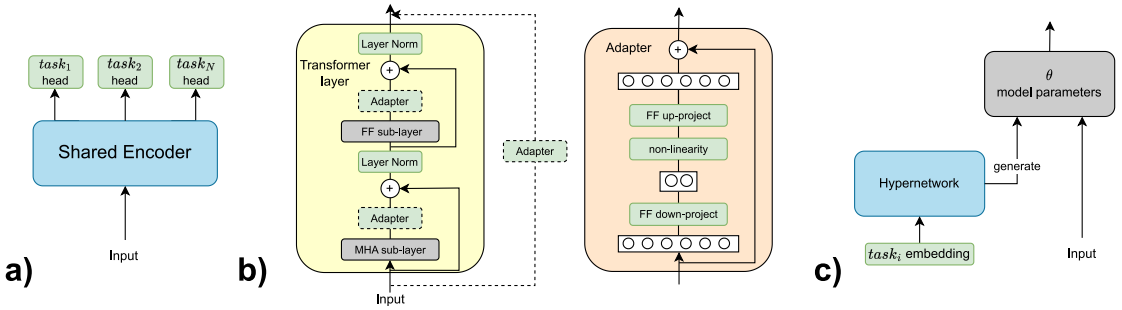


Fig. 1. Simplified overview of MTL architectures. Sub-figure **(a)** represents a fully-shared encoder, **(b)** an adapter, and **(c)** a hypernetwork. Blue components are trained jointly by all the tasks, green ones are task-specific, and gray ones are kept frozen. A dotted adapter component suggests possible adapter insertion positions. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

introduced, which stands for parameter-efficient fine-tuning, and it serves a similar purpose.[5]

Bottleneck adapters. In AdapterHub's documentation, three different bottleneck adapter approaches were mentioned. Adapters can be inserted after both Multi-Head Attention (MHA) and Feed-Forward (FF) block (Houlsby et al., 2019), only after the FF block (Pfeiffer et al., 2020c), or in parallel to the Transformer layers (He et al., 2021b). Bottleneck adapters consist of a down-projection, non-linearity (typically ReLU), and an up-projection back to the original size. Residual connection is used, and layer normalization is applied afterward.

Language adapters. In the MAD-X framework (Pfeiffer et al., 2020c), the authors train (1) *language adapters* via masked language modeling (MLM) on unlabeled target language data, and (2) *task adapters* by optimizing a target task on labeled data in a source language with the most training data. Then, adapters are stacked, allowing for a zero-shot cross-lingual transfer by substituting the target language adapter at inference. Invertible adapters are introduced to tackle the mismatch between the pre-trained model's multilingual vocabulary and target language vocabulary. Consequently, language adapters could be useful when one had already trained a task adapter for a specific task and now needs to perform inference for the same task, but on new data from a different language.

Other. Projected Attention Layer (PAL) (Stickland and Murray, 2019) is a low-dimensional multi-head attention layer added in parallel to the transformer layers. Multi-head attention is applied on a down-projected input, after which an up-projection to an original dimension is applied. These down- and up-projection matrices are shared among layers, but not among tasks. The authors fine-tune a pre-trained encoder along the PALs. This has downsides: (1) forgetting pre-trained knowledge is possible, (2) access to all the tasks at training time is required, and (3) adding new tasks requires complete joint retraining. Thus, this approach misses many of the characteristics of an adapter.

AdapterFusion (Pfeiffer et al., 2020a) introduces a knowledge composition phase, in which the previously trained adapters are combined. This approach uses multiple adapters to maximize knowledge transfer between tasks without suffering from the MTL drawbacks, such as catastrophic forgetting (Serra et al., 2018) or task interference (Wu et al., 2020). It introduces a new set of weights that learn to combine the adapters as a dynamic function of the target task data by using attention. This shows its greatest downside — AdapterFusion is trained for one task only.

Hu et al. (2021) argue that the original adapter bottleneck design (Houlsby et al., 2019) introduces inference latency because the adapters are processed sequentially, whereas large language models (LLMs) rely on hardware parallelism. Their approach, LoRA (Low Rank Approximation) modifies attention weights of query and value projection matrices by introducing trainable low-rank decomposition matrices in parallel to the original computation. This reduces inference latency, as the decomposition matrices can be merged with the pre-trained weights for faster inference. LoRA has become increasingly popular in recent years due to its effectiveness and inference efficiency (Yu et al., 2023; Zhang et al., 2023a).

3.2.3. *Hypernetworks*

A hypernetwork is a network that generates weights of another network (Ha et al., 2016). This approach can alleviate a downside of adapters, which is a lack of knowledge sharing. Hypernetwork allows the sharing of knowledge across tasks while adapting to individual tasks through task-specific parameter generation.

CA-MTL (Pilault et al., 2020) modularizes a pre-trained network by either adding task-conditioned layers or changing the pre-trained weights using the task embedding. Their task-conditioned transformer-based network has four components: (1) conditional attention, (2) conditional alignment, (3) conditional layer normalization, and (4) conditional bottleneck. In (1), they use block-diagonal conditional attention which allows the attention to account for task-specific biases. Component (2) aligns the data of diverse tasks. In (3), they adjust layer normalization statistics for specific tasks. Finally, (4) facilitates weight sharing and boosts task-specific information flow from lower layers. In addition, they use multi-task uncertainty sampling. This favors tasks with the highest uncertainty by sampling a task whenever its entropy increases, helping to avoid catastrophic forgetting. When introducing a new task, they claim that only a new linear decoder head and a new task embedding vector need to be added to re-modulate existing weights.

HyperFormer++ (Mahabadi et al., 2021) uses hypernetworks to generate the weights of adapter and layer normalization parameters. These hypernetworks condition on a task embedding, adapter position (after MHA or FF sub-layer), and layer id within the T5 model (Raffel et al., 2020). During training, they sample the tasks using temperature-based sampling. They state that for each new task, their model only requires learning an additional task embedding.

HyperGrid (Tay et al., 2020) leverages a grid-wise decomposable hyper projection structure which helps specialize regions in weight matrices for different tasks. To construct the proposed hypernetwork, their method learns the interactions and composition between a global, task-agnostic state and a local, task-specific state. They equip position-wise FF sub-layers of a Transformer with HyperGrid. They initialize a T5 model from a pre-trained checkpoint and add additional parameters that are fine-tuned along the rest of the network. The authors of the paper did not mention anything specific regarding the ability to add a new task without re-training, as it appears to be non-trivial.

4. MTL from ML lifecycle point of view

In this section, we highlight the challenges and advocate for opportunities of integrating MTL approaches into ML production systems, as opposed to using multiple single-task counterparts. Motivated by

[5] We refer interested readers to https://huggingface.co/PEFT.

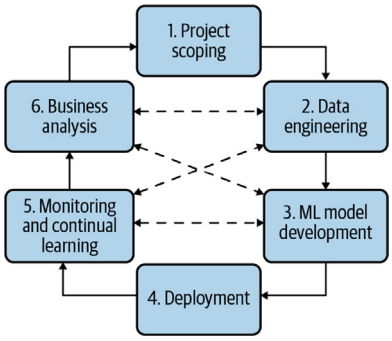


Fig. 2. ML lifecycle phases.
Source: Image is taken from Huyen (2022).

Table 2
ML lifecycle phases and corresponding challenges.

ML lifecycle phase	Challenges
Project Scoping	Initial Requirements
Data Engineering	Labeling of Large Volumes of Data Cost of Annotators and Experts Lack of High-Variance Data
Model Development	Model Complexity Resource-Constrained Environments Computational Cost Environmental Impact
Deployment	Ease of integration
Monitoring	Distribution Shift
Business Analysis	New Requirements

previous reviews on the ML lifecycle (see Section 2.2), we adopt a structured approach to define ML lifecycle phases. This framework allows us to systematically discuss the challenges and our position on opportunities of integrating MTL approaches into production systems.

Following Huyen (2022), we define six ML lifecycle phases: (1) Project Scoping, (2) Data Engineering, (3) Model Development, (4) Deployment, (5) Monitoring, and (6) Business Analysis (Fig. 2). Next, we have chosen to focus on a curated subset of challenges that were discussed in a broader scope in Paleyes et al. (2022). This decision was guided by our objective to highlight areas where we posit MTL presents a distinct advantage in addressing specific inefficiencies encountered in the ML lifecycle. The rationale behind concentrating on a subset of challenges is multifaceted. Firstly, we aim to highlight challenges that demonstrate clear, tangible benefits of MTL, thereby emphasizing its potential to significantly improve production ML systems. This includes an evidence-based selection of challenges, where empirical data validate the efficiency of MTL approaches. Furthermore, our focus is directed toward challenges with widespread implications across the ML lifecycle, aiming to illustrate the broad advantages of MTL adoption. This choice allows us to articulate a coherent narrative around MTL's capacity to mitigate specific challenges. Challenges per each phase of the ML lifecycle are listed in Table 2. Paleyes et al. (2022) listed other challenges that are equally important, besides those we have focused on and presented in Table 2. For these challenges, we do not assess whether MTL mitigates or worsens them compared to single-task counterparts, as they fall outside the scope of this paper. However, we encourage readers to consider them as well when developing ML systems. When discussing the challenges and opportunities of using MTL approaches, we compare them to corresponding single-task model solutions.

In the rest of the section, we first turn our attention to the data engineering and model development phases and their associated challenges. Then, we highlight the challenges of updating ML models, pinpointing specific aspects that introduce varied challenges across the ML lifecycle phases. For each phase, we argue for our position that MTL can alleviate the challenges that arise.

4.1. Data engineering

The first phase we discuss is data engineering. This phase focuses on preparing data that is needed to train a machine learning model, while we have a particular interest in challenges related to the lack of labeled data (see Table 2).

Challenges. The need for data augmentation can arise from various factors, with one of the most problematic being the lack of labels in the data, especially in real-world applications where labeled data may be scarce. It is a common practice in NLP production systems to utilize pre-trained transformer-based language models by fine-tuning them for specific downstream tasks. However, if there is a significant gap between a downstream task and the pre-training objectives, a larger amount of labeled data may still be required to achieve the target performance (Wang et al., 2020). Obtaining this data involves costly and time-consuming involvement of annotators and domain experts. Additionally, the absence of high-variance data results in a model that is unable to generalize well, such as adapting language models to low-resource languages (Clark et al., 2019b).

Opportunities. We argue that the challenges posed by the lack of labeled data could be alleviated using MTL approaches. For example, if a set of single-task models is being used in a production system, training an MTL model instead can help alleviate data sparsity by jointly learning to solve related tasks (Section 3.2.1). The benefits of MTL have been previously discussed in Caruana (1997), Ruder (2017), including its ability to increase data-efficiency. First, different tasks transfer different knowledge aspects to each other, enhancing the representation's ability to express the input text, which can be beneficial for tasks with low-resource datasets (Section 3.2.1). However, some MTL approaches may underperform in resource-constrained environments due to inadequate optimization choices (Section 3.2.1). Additionally, the presence of different noise patterns in each task acts as an implicit data augmentation method, effectively increasing the sample size used for training, and leading to a robust model with more general representations (Ruder, 2017). Using pre-finetuning (Section 3.2.1) could also reduce convergence time, saving computational resources. Finally, adapters (Section 3.2.2) have even been shown to improve performance over full fine-tuning when dealing with low-resource datasets (He et al., 2021a). A real-world example is the Amazon's team which solved the data scarcity problem by using learned embeddings from related tasks to improve the accuracy of Alexa when predicting graph-based semantic representations, such as fine-grained types, properties, actions, and roles (Perera et al., 2018). Palma et al. (2021) applied a multi-task learning approach to train a risk prediction model for the Chilean National Geology and Mining Service, which is responsible for ensuring mine safety, effectively overcoming data availability constraints by fusing different sources to predict mining industry accidents. Furthermore, compared to single-task baselines, Google's robotics team model MT-Opt performed similarly on tasks with the most data and significantly improved performance on underrepresented tasks (Kalashnikov et al., 2021). For a generic lifting task, which had the most supporting data, MT-Opt achieved an 89% success rate, compared to 88% for QT-Opt baseline, while it achieved a 50% average success rate across rare tasks, compared to 1% with a single-task QT-Opt baseline. The opportunities highlighted here reinforce our stance on the advantages of employing MTL during the data engineering stage of an ML lifecycle.

4.2. Model development

In the model development phase, we focus on two groups of challenges. The first group refers to the *model selection* problem, including issues related to model complexity and resource constraints. The second

group of challenges is related to problems in *model training*, such as the computational cost of the training procedure and its impact on the environment. We refer readers to Gupta and Agrawal (2022) for an overview of methods for efficient models in text.

Model Selection Challenges. When selecting a model to handle tasks that end-users are interested in, practitioners often face a dilemma regarding the trade-off between model complexity and performance. Typically, complex models have better performance, but they come with the risk of over-complicating the design in the first place, leading to a longer development time and deployment failure (Haldar et al., 2019). Furthermore, they may not be practical to use in resource-constrained environments where they require high computational and memory resources.

Model Selection Opportunities. We argue that MTL architectures, presented in Section 3.2, have properties that can alleviate challenges related to the trade-off between model complexity and performance. For example, consider replacing N single-task models with a single MTL shared encoder model. The MTL model will have close to N times smaller memory footprint, as the number of task-specific parameters is negligible compared to the number of shared parameters. This reduction results in a better fit to memory-constrained environments while only having slightly worse performance than the single-task counterparts. For example, Meta's GrokNet, a deployed image recognition system for commerce applications, significantly enhanced exact product match accuracy by 2.1 times compared to Meta's previous product recognition system (Bell et al., 2020). This improvement was achieved through a multi-task learning approach, utilizing a single computer vision trunk, i.e. shared encoder, to train on 7 datasets across various commerce domains. GrokNet has demonstrated gains in production applications and operated at Meta's scale. With such a reduction in the number of parameters, the system's complexity drops accordingly. Similarly, training and saving N adapters or a single hypernetwork is much more memory-efficient than saving N single-task models. Adapters and hypernetworks have also been shown to match or even improve the performance compared to in-context learning of models with billions of parameters (Liu et al., 2022) or the expensive fine-tuning of the whole model (Pilault et al., 2020), especially for low-resource datasets (He et al., 2021a). Although the transformer-based solutions suffer in interpretability, working with only a fraction of trainable parameters through adapters, around 0.2%, could make the gradients more interpretable during training (Hu et al., 2021). Additionally, task or language embeddings trained as part of a hypernetwork (Üstün et al., 2022) could later be utilized for comparing the similarity of different tasks or languages. All of the aforementioned points elucidate why MTL offers significant advantages during the model selection phase.

Model Training Challenges. The training of machine learning models presents several challenges that must be addressed by practitioners. One of the major challenges is the high economic cost associated with training, which is due to the computational resources required. In the field of natural language processing, the cost of model training continues to rise, even as the cost of individual floating-point operations decreases, due to factors such as the growth in training dataset size, number of model parameters, and number of operations involved in the training process (Sharir et al., 2020). The training process also has a significant impact on the environment, leading to increased energy consumption and greenhouse gas emissions (Strubell et al., 2020). These challenges emphasize the need to address the economic and environmental implications of training machine learning models.

Model Training Opportunities. Computational resource challenges could be alleviated in some aspects by using MTL approaches to reduce the cost of model training. First, in certain cases, smaller dataset sizes can be used due to pre-finetuning or knowledge transfer between related tasks during joint training of the MTL model, leading to more data-efficient training. Second, the joint model is more parameter-efficient, resulting in a significant reduction in the number of parameters required for multiple single-task models. Both arguments generally lead to a reduction in floating-point operations (FLOPs) and eliminate the need for a GPU with extensive VRAM. A real-world example of training a model using multi-task learning is Amazon's Alexa. It uses multi-task learning to jointly learn tasks of predicting the spoken language understanding (SLU) or Alexa Meaning Representation Language (AMRL) representations (Perera et al., 2018). This allowed them to explore commonalities between tasks such as overlapping spans between AMRL types and properties. Furthermore, the Microsoft Vision Model ResNet-50, developed by the Multimedia Group at Microsoft Bing, employed multi-task learning with hard parameter sharing to pre-train the MTL model across four datasets to reduce the cost of performing downstream tasks in production (Lenyk and Park, 2021). It achieved state-of-the-art performance for image and visual search applications and was used in Microsoft's Image Search and Visual Search. Zhang et al. (2020) applied MTL to forecast electricity, heat, and gas loads in an industrial-park integrated energy system and reported MTL efficiency in reducing training time. The authors reported that although training for each individual energy source in a single-task setting took less time, the total time required for multitask learning was less than the combined sum of all the single-task training times.

Selection, Training, and Inference Trade-Off. Here, we delve into the specifics of selecting different MTL architectures, training them, and using them for inference. To start with, the number of FLOPs is not reduced in some cases and depends on the choice of MTL architecture and the nature of the tasks. Different task-specific heads require different inputs if tasks belong to different domains or have different input encodings. However, if tasks belong to the same domain and have the same input encoding, part of the computation can be shared among the task-specific heads. For example, in a fully-shared encoder (Section 3.2.1), the computation of the full encoder can be shared, while the computation in the task-specific heads is negligible. Similarly goes for adapters (Section 3.2.2) — the large majority of the computation is shared, and only the adapters are then dynamically plugged for performing different tasks on the same input. The mentioned computation sharing holds for both training and inference. A practical example in the real world could involve using the same social media content as input and connecting different adapters or heads to perform various tasks, such as (1) detecting inappropriate content, (2) categorizing content into pre-defined topic categories, (3) identifying fake news, (4) detecting language, etc. This approach requires deploying a single encoder and N adapters/heads, where N is the number of tasks to be performed. This results in a significant reduction in inference time, as an example only passes through the encoder once, instead of N times. Namely, the benefit of using adapters is a small number of trainable parameters, thanks to a frozen encoder, which results in a faster gradient back-propagation and ultimately — faster training. A forward pass, on the other hand, takes more time compared to an encoder with no adapters. Thus, when tasks do not share the inputs, using adapters results in a slightly longer inference time compared to single-task counterparts. AdapterFusion inference is even slower, as an input must pass through all the available adapters (Rücklé et al., 2020). LoRA solves the inference latency problem by using parameter composition (Hu et al., 2021). Systems can use InfAdapter (Salmani et al., 2023) for inference during serving in high-load scenarios where low latency is crucial. The InfAdapter approach dynamically resizes the resources (auto-scaling) and switches between higher and lower model complexity/performance (model-switching). During the high load times, the system could switch to a less complex version of a model, such as adapters, to keep the throughput constantly high. The downside is that this would require training both more and less complex model versions in order to switch between them dynamically.

4.3. Model deployment

In the model deployment phase, our focus is on simplifying the integration of trained ML models with existing ML systems that are running in production, with a particular emphasis on straightforward implementation, seamless collaboration, and ease of maintenance.

Challenges. The first challenge in model deployment is preparing the developed model for use in a production environment. The initial versions of models are often developed by stakeholders (e.g., ML researchers) who are different from those responsible for deploying them into production (e.g., ML and DevOps engineers). This means that the model code needs to be adapted to meet the requirements of the operational production environment. Those requirements are typically more strict and different from those available during the development phase, so it is important to address operational aspects such as scalability, security, and reliability. Hence, each additional model adds complexity to the process, both for the stakeholders involved and for the ML system infrastructure and computational resources in place. Another challenge during model integration is incorporating the model into real-data processing pipelines in production, whether it is for batch offline processes or handling real-time user requests. During model training, the researcher often uses preprocessed and clean datasets. However, when integrating the model into production, it will be integrated into existing data pipelines. The more complex the model's input data requirements, or the more models that are used, the more complex the data processing pipelines will need to be. Adapting the models and existing data pipelines often requires collaboration between different stakeholders or teams responsible for different parts of the ML system and/or ML lifecycle phase. All of these factors directly impact the increasing costs of maintenance, operations, support, and infrastructure for model deployment.

Opportunities. We argue that the MTL approach, as exemplified by the team from Pinterest, presents a significant opportunity to streamline and enhance machine learning pipelines. In their work, a universal set of image embeddings was trained for three different models, resulting in simplified deployment pipelines and improved performance on individual tasks (Zhai et al., 2019). The cornerstone of our position is the efficiency MTL introduces: it significantly reduces the number of parameters necessary for supporting diverse models tasked with solving varied problems. This efficiency leads to the development of a single, smaller MTL model, instead of having multiple single-task counterparts, diminishing the need for extensive code adjustments within production environments and lowering the overhead associated with changes in existing data pipelines.

Next, the modular design inherent in MTL architectures, such as the fully-shared encoder architecture (Section 3.2.1), facilitates the seamless integration of models into production environments. This approach enables the reuse of a common encoder across multiple task-specific heads, streamlining the deployment process by reducing the need for extensive code modifications when models are updated or new task-specific heads are introduced. Additionally, freezing the shared encoder, as employed in the HydraNet MTL model by the Tesla AI team (Karpathy, 2021), facilitates deployment further by allowing independent updates to task-specific heads without affecting the shared components, although it is important to consider the potential performance drop this approach might entail (Section 3.2.1). Furthermore, the AdapterHub framework (described in Section 3.2.2) allows for the dynamic insertion or replacement of adapter modules into pre-trained models, offering a flexible and modular solution for adapting and deploying machine learning models in production environments. This concept acts as a distributed MTL (Pfeiffer et al., 2023). As advocated in Akoush et al. (2022), engineers may deploy adapters or task-specific heads in production (e.g., through Kubernetes) using the multimodel serving pattern.[6] In this approach, a single ML inference server hosts multiple models per server instead of only one. In the case of MTL, these models could consist of adapters or task-specific heads and a Transformer backbone. A consequence is reduced overheads required to deploy numerous models individually while simplifying operating the system at scale.

To conclude, this exemplifies how the modularity of MTL architectures can significantly ease the integration and maintenance of ML systems in the deployment phase, underscoring our position on the benefits of MTL in simplifying the deployment of machine learning models.

4.4. Model updating through multiple ML lifecycle phases

Often it is necessary to update the ML model regularly after it has been deployed and is running in production, in order to keep it aligned with the most current changes in data and environment. The need for updating models is one of the most important requirements of ML production systems (Pacheco et al., 2018; Abdelkader, 2020; Lakshmanan et al., 2020; Paleyes et al., 2022; Huyen, 2022; Wu and Xie, 2022; Nahar et al., 2022). In this section, we address the challenges of model updating and argue how MTL approaches can alleviate these challenges. Challenges of different phases of the ML lifecycle can trigger the need for model updates. First, we identify these occurrences and elaborate on the actions they trigger. Then, we argue how MTL approaches can alleviate these challenges and highlight the current limitations of these approaches.

Model Updating Challenges. A distribution shift is one of the common reasons for the need to update models. Distribution shift refers to changes observed in the joint distribution of the input and output variables of an ML model (Ditzler et al., 2015). Two problems must be solved to address the distribution shift effectively. First, in the *monitoring phase*, a mechanism must be in place to detect changes in distribution or a drop in key performance indicators, which will signal the need for an ML model update. Second, in the *model development phase*, there must be a way to continuously learn and update the model in response to signals from the monitoring phase.

New business requirements often arise, requiring the model to have new capabilities in addition to the existing ones. For example, a named entity recognition model trained for 10 entity labels in news articles may require 5 new labels after 3 months of use. New requirements are introduced during the *business analysis phase*, which also triggers the need for an ML model update. Unlike the initial requirements in the *project scoping phase*, adding new requirements should be possible without re-training the full model.

Periodic or scheduled re-training and continual learning are the most common approaches for adapting models to new data and requirements. Periodic re-training refers to the process of re-training a model at a predetermined interval, regardless of whether there have been any changes to the data or the environment. The frequency of updating is determined beforehand and is typically based on the amount of data and the desired level of model performance. Striking a balance between updating the model regularly to maintain good performance and avoiding redundant model updates that bring no performance improvement to minimize computational costs is crucial. It is important to note that different models may require different re-training schedules, making it a situation-dependent problem. Finding the optimal re-training schedule requires careful consideration of the specific requirements and circumstances of each ML system. Continual learning, on the other hand, refers to the ability of an ML model to adapt to changes in the data and environment over time. This approach involves continuously monitoring the performance of the model and updating it as needed to ensure it remains accurate and up-to-date. The frequency of model updates in continual learning is determined dynamically based on the changes observed in the data and environment.

Model Updating Opportunities. Following Huyen (2022), we differentiate two types of model updates: *data iteration* and *model iteration*.

[6] https://kubernetes.io/

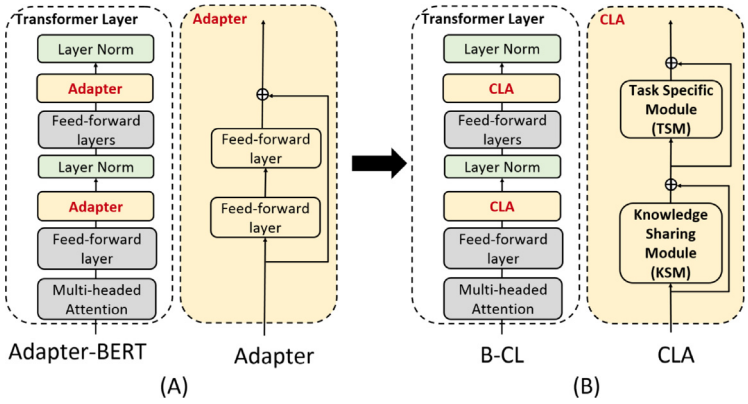


Fig. 3. (A). Adapter-BERT (Houlsby et al., 2019) uses adapters in a transformer layer (Vaswani et al., 2017). Adapters are 2-layer networks with skip-connections, added twice per layer. Only adapters (yellow) and layer norm (green) are trainable while other modules (gray) are frozen. **(B).** B-CL replaces adapters with CLA, containing a knowledge-sharing module (KSM) and task-specific module (TSM), both with skip-connections. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)
Source: Image and modified caption are taken from Ke et al. (2021b).

Data iteration refers to updating the model with new data while keeping the model architecture and features the same, while model iteration refers to adding new features to an existing model architecture or changing the model architecture itself. To argue for our position on the opportunities of how MTL approaches could alleviate the challenges of model updating, we examine two distinct scenarios. First, periodic re-training is done every 6 or 12 months, with the goal of training each model to perform optimally using all available data. Due to the high economic cost, we assume this cannot be done more frequently. Second, between periodic retrains, we assume there may be situations where data iteration is necessary due to a distribution shift, or model iteration is required to extend model capabilities in response to a new business requirement.

We assert that incorporating MTL approaches into the update scenario would be practical, in addition to the benefits discussed in Sections 4.1–4.3. We argue that MTL is particularly well-suited for periodic re-training scenarios where the objective is to obtain a single best-performing model on all tasks using all available data, instead of training individual models for each task. However, the challenge lies in how MTL approaches could manage the second scenario where an MTL model must learn new tasks or domains sequentially over time. We advocate for the necessity of a model's capability to be able to learn in both MTL and CL settings to effectively handle the challenges of model updating, and we refer to this setting as Continual MTL (CMTL).

Advocating for Continual MTL. MTL and CL models both learn multiple tasks, however, MTL learns them simultaneously, while CL learns them incrementally. As mentioned in Section 2.3, the authors of Sun et al. (2020) combined CL and MTL to further improve pre-training. Although the approach enhanced performance on downstream tasks, it does not address the real-world scenario in which an MTL model should be updated to support new downstream tasks or handle distribution shifts. Moreover, the total number of sequential tasks must be known a priori for the algorithm to determine an efficient training schedule. We argue that the similarities between MTL and CL architectures could enable the construction of a CMTL model. For example, most MTL architectures in Section 3.2 are transformer-based MTL architectures with task-specific heads, which resemble the parameter-isolation architectures for the TIL setting in CL (Section 2.3). In TIL, the task identifier is available during both training and testing and is used to identify task-specific parameters in multi-headed architectures, similar to MTL architectures (Ke and Liu, 2022). This similarity can be seen in the use of adapter architectures (Section 3.2.2) in both MTL (Stickland and Murray, 2019; Pfeiffer et al., 2020c; He et al., 2021b) and CL (Ke et al., 2021a,b), as well as in the use of hypernetworks (Section 3.2.3) in both MTL (Pilault et al., 2020; Mahabadi et al., 2021) and CL (Von Oswald et al., 2019; Jin et al., 2021).

For example, if we consider the BERT layer in Fig. 3, we can observe that the adapter layers for the MTL approach in Fig. 3(A) are in the same positions as the CL adapters in Fig. 3(B) (Ke et al., 2021b). We posit that this alignment suggests that transitioning from MTL to CL could be as straightforward as changing the adapters. We assume this structural similarity suggests a potential avenue for exploring research into CMTL models. However, the viability and performance of these models depend on the presence of a comprehensive benchmark for detailed assessment. Developing such a benchmark could also be a part of future research work.

5. Conclusion

In this position paper, we have articulated a stance on the opportunities of integration of transformer-based MTL approaches in NLP production systems, emphasizing their potential to alleviate the challenges encountered across various phases of the ML lifecycle. Our position underscored the advantage of MTL in streamlining data engineering, model development, deployment, and monitoring phases, offering a more holistic and efficient solution compared to the conventional reliance on multiple single-task models. Further, we delved into the critical role of MTL in addressing the pressing need for model updates, driven by distribution shifts and the evolution of real-world requirements. The inherent capability of MTL to facilitate simultaneous learning of multiple tasks positions it as an efficient approach for periodic model retraining. However, we have also highlighted the limitations of existing MTL approaches in managing sequential updates between periodic retraining, a setting where CL proves to be more effective. To this end, we have hypothesized the concept of the CMTL setting and posited a unified model capable of handling both MTL and CL scenarios. Although this hypothesis lacks extensive empirical support, we hope it will motivate further investigation into the practical implementation and efficacy of such a model, which remains a subject for future research.

In conclusion, our paper posits that MTL could improve the efficiency of ML systems throughout the ML lifecycle. Furthermore, we hypothesize that this improvement could be extended if MTL is augmented with the principles of CL in the CMTL setting. Our stance is that MTL should be recognized as an important component of practitioners' toolbox seeking to alleviate the challenges identified within their ML systems, as discussed throughout this position paper.

CRediT authorship contribution statement

Lovre Torbarina: Writing – original draft, Validation, Supervision, Investigation, Conceptualization. **Tin Ferkovic:** Writing – original

draft, Visualization, Investigation, Conceptualization. **Lukasz Roguski:** Writing – review & editing, Validation, Conceptualization. **Velimir Mihelcic:** Writing – review & editing, Validation. **Bruno Sarlija:** Writing – review & editing, Validation. **Zeljko Kraljevic:** Writing – review & editing, Validation, Supervision, Project administration, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Lovre Torbarina reports a relationship with doXray that includes: employment. Tin Ferkovic reports a relationship with doXray that includes: employment. Lukasz Roguski reports a relationship with doXray that includes: employment. Velimir Mihelcic reports a relationship with doXray that includes: employment. Bruno Sarlija reports a relationship with doXray that includes: employment. Zeljko Kraljevic reports a relationship with doXray that includes: employment. If there are other authors,r they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Additional multi-task learning approaches

A.1. Prompts

Prompts embed a task in the input. The original input $\mathbf{x}$ is modified using a template into a textual string prompt $\mathbf{x}'$ that has some unfilled slots, and then the LM is used to fill the information to obtain a final string $\hat{\mathbf{x}}$, from which the final output $\mathbf{y}$ can be derived (Liu et al., 2021a). Prompting requires redesigning all the inputs and outputs in order to treat the tasks as text-to-text problems. The prompting approach proved to work the best in the zero- and few-show scenarios, but the benefits dissipate in the high-resource settings (Parmar et al., 2022; Wang et al., 2022). Also, performance scales well with an increase in model size (Lester et al., 2021), making this approach not accessible to everyone.

According to Liu et al. (2021a), there are many flavors to prompting. First, models with different pre-training objectives can be used — left-to-right LM (Brown et al., 2020), MLM (Liu et al., 2019c), prefix LM (Bao et al., 2020), or encoder–decoder one (Lewis et al., 2020). Prompts can be engineered in a cloze (Petroni et al., 2019) or prefix (Lester et al., 2021) shape, be hand-crafted (Brown et al., 2020) or automated, which can again be discrete (Shin et al., 2020) or continuous (Lester et al., 2021). Answer engineering searches for an answer space and a mapping to the original output by deciding the answer shape and choosing an answer design method. Furthermore, parameters can be updated using different settings — tuning-free prompting (Brown et al., 2020), fixed-LM prompt tuning (Li and Liang, 2021), fixed-prompt LM tuning (Raffel et al., 2020), and prompt+LM tuning (Liu et al., 2021b). Finally, training can be done in a few/zero-shot (Brown et al., 2020) or full-data setting (Han et al., 2021). In the following paragraphs, we discuss some of the previous prompting works.

HyperPrompt (He et al., 2022) is an approach that combines hypernetworks and prompts. The key idea is to prepend task-conditioned trainable vectors to both keys and values of MHA sub-layer at every Transformer layer. These task-specific attention feature maps are jointly trained with the task-agnostic representations. Key prompts interact with the original query, enabling tokens to acquire task-specific semantics. Value prompts are prepended to the original value vector, serving as task-specific memories for MHA to retrieve information from. However, instead of having a key/value prompt for each task and layer, authors initialize a global prompt P for each task. They apply two local hypernetworks $h_{k/v}$ (one for keys, the other for values) at each Transformer layer in order to project this prompt into actual task- and layer-specific key/value prompts. There are three variations examined in the paper — HyperPrompt-Share, -Sep, and -Global. HyperPrompt-Global proved to be the best, as it allows a flexible way to share information across tasks and layers. It introduces task and layer embeddings, which are then fused into the layer-aware task embedding. This embedding is then an input for the global hypernetworks $H_{k/v}$ used as a generator of local hypernetworks $h_{k/v}$. These local hypernetworks then generate hyper-prompts using a global prompt P. Hyper-prompts are finally prepended with the original keys and values in the MHA sub-layers. During training, no parameters are kept frozen. They report better results compared to the competitive methods of HyperFormer++ (Mahabadi et al., 2021) and Prompt-Tuning (Lester et al., 2021).

In-BoXBART (Parmar et al., 2022) uses biomedical instructions which contain: (1) definition (core explanation and detailed instructions about what needs to be done), (2) prompt (short explanation of the task), and (3) examples (input/output pairs with the explanation). Each instruction (effectively a prompt) is prepended to the input instances. A problem of too long instances arises.

InstructionNER (Wang et al., 2022) enriches the inputs with task-specific instructions and answer options. Additionally, two auxiliary tasks are introduced — *entity extraction* and *entity typing*, which directly help in solving a NER task.

Sanh et al. (2021) allowed public to suggest prompts and came up with 2073 prompts for 177 datasets in total (12 prompts per dataset on average). They shuffled and combined all the examples from the datasets prior to training. In most of the cases, performance of their models improved as the number of training datasets increased. Moreover, training on more prompts per dataset resulted in a better and more robust generalization on unseen datasets. The models they trained are based on LM-adapted T5 model (Lester et al., 2021).

Prefix tuning freezes the language model parameters and optimizes a small, continuous task-specific vector called *prefix* (Li and Liang, 2021). Consequently, only a prefix needs to be stored for each task, making prefix tuning modular and space-efficient. This approach can be applied solely to text generation models, such as GPT-2 (Radford et al., 2019) and BART (Lewis et al., 2020). They state the intuition that the context can influence the encoding of input $\mathbf{x}$ by guiding what to extract from $\mathbf{x}$; and can influence the generation of output $\mathbf{y}$ by steering the next token distribution. They optimize the prefix as continuous word embeddings, instead of optimizing over discrete tokens. The reason for this is that the discrete prompt needs to match the real word embedding, resulting in a less expressive model.

Lester et al. (2021) used a fixed-length prompt of special tokens, where the embeddings of these tokens are updated. This removes the requirement that the prompts are parametrized by the model and allows them to have their own trainable parameters. They test three different initialization techniques — random uniform, sampled vocabulary, and class label. LM-adaptation pre-training technique was used. Unlike adapters, which modify the actual function that acts on the input, prompt tuning adds new input representations that affect how input is processed, leaving the function fixed. They freeze the pre-trained model. Finally, they try prompt ensembling (one prompt per model, for each task), which showed improved performance compared to a single-prompt average.

Challenges and Opportunities in ML Lifecycle. Prompts can also mitigate some of the challenges of different ML lifecycle phases (see Table 2). To start with, in a fixed-prompt LM tuning or a prompt+LM tuning setting, knowledge is transferred from other tasks, which can be especially beneficial for low-resource tasks. However, in such a scenario, different optimization techniques need to be considered in order to avoid negative interference and other MTL drawbacks (Section 2.1). As for the model selection, it is more of a challenge than an opportunity, as models based on prompts require large number of parameters to begin with in order to perform well. Model training complexity depends on the parameter update setting, with some settings

requiring no (Brown et al., 2020) or only few parameter updates (Li and Liang, 2021). One of the benefits when using prompts is the ability to handle all tasks using a single text-to-text model, regardless of the input and output encoding. When using prompts, the biggest challenge in model deployment is the model's size. Finally, model updating due to distribution shift, new data, or business requirements (see Section 4.4) seems most plausible in the setting where prompts are continuous and tuned (Li and Liang, 2021). However, this has downsides, such as difficult optimization, non-monotonic performance change with regard to the number of parameters, and reserving a part of sequence length for adaptation (Hu et al., 2021).

References

Abdelkader, H., 2020. Towards robust production machine learning systems: Managing dataset shift. In: 2020 35th IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 1164–1166.

Abhadiomhen, S.E., Nzeh, R.C., Ganaa, E.D., Nwagwu, H.C., Okereke, G.E., Routray, S., 2022. Supervised shallow multi-task learning: analysis of methods. Neural Process. Lett. 54 (3), 2491–2508.

Aghajanyan, A., Gupta, A., Shrivastava, A., Chen, X., Zettlemoyer, L., Gupta, S., 2021. Muppet: Massive multi-task representations with pre-finetuning. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. pp. 5799–5811.

Akoush, S., Paleyes, A., Van Looveren, A., Cox, C., 2022. Desiderata for next generation of ML model serving. arXiv preprint arXiv:2210.14665.

Aribandi, V., Tay, Y., Schuster, T., Rao, J., Zheng, H.S., Mehta, S.V., Zhuang, H., Tran, V.Q., Bahri, D., Ni, J., et al., 2021. ExT5: Towards extreme multi-task scaling for transfer learning. In: International Conference on Learning Representations.

Ashmore, R., Calinescu, R., Paterson, C., 2021. Assuring the machine learning lifecycle: Desiderata, methods, and challenges. ACM Comput. Surv. 54 (5), 1–39.

Bao, H., Dong, L., Wei, F., Wang, W., Yang, N., Liu, X., Wang, Y., Gao, J., Piao, S., Zhou, M., Hon, H.W., 2020. UniLMv2: Pseudo-masked language models for unified language model pre-training. In: III, H.D., Singh, A. (Eds.), Proceedings of the 37th International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 119, PMLR, pp. 642–652, URL: https://proceedings.mlr.press/v119/bao20a.html.

Bell, S., Liu, Y., Alsheikh, S., Tang, Y., Pizzi, E., Henning, M., Singh, K., Parkhi, O., Borisyuk, F., 2020. GrokNet: Unified computer vision model trunk and embeddings for commerce. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 2608–2616.

Bernardi, L., Mavridis, T., Estevez, P., 2019. 150 successful machine learning models: 6 lessons learned at booking. com. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 1743–1751.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020. Language models are few-shot learners. CoRR abs/2005.14165. URL: https://arxiv.org/abs/2005.14165.

Cabrera, C., Paleyes, A., Thodoroff, P., Lawrence, N.D., 2023. Real-world machine learning systems: A survey from a data-oriented architecture perspective. arXiv preprint arXiv:2302.04810.

Caruana, R., 1997. Multitask learning. Mach. Learn. 28 (1), 41–75.

Chen, Z., Liu, B., 2018. Lifelong machine learning. Synth. Lect. Artif. Intell. Mach. Learn. 12 (3), 1–207.

Chen, S., Zhang, Y., Yang, Q., 2021. Multi-task learning in natural language processing: An overview. arXiv preprint arXiv:2109.09138.

Chui, M., Hall, B., Mayhew, H., Singla, A., 2022. The State of AI in 2022–and a Half Decade in Review. McKinsey & Company, URL: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-in-2022-and-a-half-decade-in-review. (Accessed 15 January 2023).

Chui, M., Hall, B., Mayhew, H., Singla, A., 2023. The State of AI in 2023–Generative AI's Breakout Year. McKinsey & Company, URL: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-in-2023-generative-ais-breakout-year. (Accessed 25 March 2024).

Chung, H.W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., Li, E., Wang, X., Dehghani, M., Brahma, S., et al., 2022. Scaling instruction-finetuned language models. arXiv preprint arXiv:2210.11416.

Clark, K., Luong, M., Khandelwal, U., Manning, C.D., Le, Q.V., 2019a. BAM! Born-again multi-task networks for natural language understanding. CoRR abs/1907.04829. URL: http://arxiv.org/abs/1907.04829.

Clark, C., Yatskar, M., Zettlemoyer, L., 2019b. Don't take the easy way out: Ensemble based methods for avoiding known dataset biases. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. EMNLP-IJCNLP, Association for Computational Linguistics, Hong Kong, China, pp. 4069–4082. http://dx.doi.org/10.18653/v1/D19-1418, URL: https://aclanthology.org/D19-1418.

Crawshaw, M., 2020. Multi-task learning with deep neural networks: A survey. arXiv preprint arXiv:2009.09796.

De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., Slabaugh, G., Tuytelaars, T., 2021. A continual learning survey: Defying forgetting in classification tasks. IEEE Trans. Pattern Anal. Mach. Intell. 44 (7), 3366–3385.

Devlin, J., Chang, M.W., Lee, K., Toutanova, K., 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Association for Computational Linguistics, Minneapolis, Minnesota, pp. 4171–4186. http://dx.doi.org/10.18653/v1/N19-1423, URL: https://aclanthology.org/N19-1423.

Ding, J., Tarokh, V., Yang, Y., 2018. Model selection techniques: An overview. IEEE Signal Process. Mag. 35 (6), 16–34.

Ditzler, G., Roveri, M., Alippi, C., Polikar, R., 2015. Learning in nonstationary environments: A survey. IEEE Comput. Intell. Mag. 10 (4), 12–25.

Fortune Business Insights, 2024. Natural language processing — Market size, share & industry analysis. https://www.fortunebusinessinsights.com/industry-reports/natural-language-processing-nlp-market-101933. (Accessed 25 March 2024).

González-Garduño, A., Søgaard, A., 2018. Learning to predict readability using eye-movement data from natives and learners. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 32, No. 1. http://dx.doi.org/10.1609/aaai.v32i1.11978, URL: https://ojs.aaai.org/index.php/AAAI/article/view/11978.

Google, 2024. Google cloud, introduction to AI platform. https://cloud.google.com/ai-platform/docs/technical-overview. (Accessed 28 March 2024).

Guo, M., Haque, A., Huang, D.A., Yeung, S., Fei-Fei, L., 2018. Dynamic task prioritization for multitask learning. In: Proceedings of the European Conference on Computer Vision. ECCV.

Guo, H., Pasunuru, R., Bansal, M., 2019. AutoSeM: Automatic task selection and mixing in multi-task learning. CoRR abs/1904.04153. URL: http://arxiv.org/abs/1904.04153.

Gupta, M., Agrawal, P., 2022. Compression of deep learning models for text: A survey. ACM Trans. Knowl. Discov. Data (TKDD) 16 (4), 1–55.

Ha, D., Dai, A., Le, Q.V., 2016. HyperNetworks. http://dx.doi.org/10.48550/ARXIV.1609.09106, URL: https://arxiv.org/abs/1609.09106.

Haldar, M., Abdool, M., Ramanathan, P., Xu, T., Yang, S., Duan, H., Zhang, Q., Barrow-Williams, N., Turnbull, B.C., Collins, B.M., et al., 2019. Applying deep learning to airbnb search. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 1927–1935.

Han, X., Zhao, W., Ding, N., Liu, Z., Sun, M., 2021. PTR: Prompt tuning with rules for text classification. CoRR abs/2105.11259. URL: https://arxiv.org/abs/2105.11259.

He, R., Liu, L., Ye, H., Tan, Q., Ding, B., Cheng, L., Low, J.W., Bing, L., Si, L., 2021a. On the effectiveness of adapter-based tuning for pretrained language model adaptation. arXiv preprint arXiv:2106.03164.

He, Y., Zheng, S., Tay, Y., Gupta, J., Du, Y., Aribandi, V., Zhao, Z., Li, Y., Chen, Z., Metzler, D., Cheng, H.T., Chi, E.H., 2022. HyperPrompt: Prompt-based task-conditioning of transformers. In: Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., Sabato, S. (Eds.), Proceedings of the 39th International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 162, PMLR, pp. 8678–8690, URL: https://proceedings.mlr.press/v162/he22f.html.

He, J., Zhou, C., Ma, X., Berg-Kirkpatrick, T., Neubig, G., 2021b. Towards a unified view of parameter-efficient transfer learning. arXiv preprint arXiv:2110.04366.

Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., Gelly, S., 2019. Parameter-efficient transfer learning for NLP. In: International Conference on Machine Learning. PMLR, pp. 2790–2799.

Hsu, Y.C., Liu, Y.C., Ramasamy, A., Kira, Z., 2018. Re-evaluating continual learning scenarios: A categorization and case for strong baselines. arXiv preprint arXiv:1810.12488.

Hu, B.C., Chechik, M., 2023. Towards feature-based analysis of the machine learning development lifecycle. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 2087–2091.

Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., 2021. LoRA: Low-rank adaptation of large language models. http://dx.doi.org/10.48550/ARXIV.2106.09685, URL: https://arxiv.org/abs/2106.09685.

Huyen, C., 2022. Designing Machine Learning Systems. " O'Reilly Media, Inc.".

Jean, S., Firat, O., Johnson, M., 2019. Adaptive scheduling for multi-task learning. CoRR abs/1909.06434. URL: http://arxiv.org/abs/1909.06434.

Jin, X., Lin, B.Y., Rostami, M., Ren, X., 2021. Learn continually, generalize rapidly: lifelong knowledge accumulation for few-shot learning. arXiv preprint arXiv:2104.08808.

Kaiser, L., Gomez, A.N., Shazeer, N., Vaswani, A., Parmar, N., Jones, L., Uszkoreit, J., 2017. One model to learn them all. http://dx.doi.org/10.48550/ARXIV.1706.05137, URL: https://arxiv.org/abs/1706.05137.

Kalashnikov, D., Varley, J., Chebotar, Y., Swanson, B., Jonschkowski, R., Finn, C., Levine, S., Hausman, K., 2021. Mt-opt: Continuous multi-task robotic reinforcement learning at scale. arXiv preprint arXiv:2104.08212.

Karpathy, A., 2021. HydraNets - Tesla AI Day 2021. URL: https://www.youtube.com/watch?t=4284&v=j0z4FweCy4M&feature=youtu.be. (Accessed 15 February 2023).

Ke, Z., Liu, B., 2022. Continual learning of natural language processing tasks: A survey. arXiv preprint arXiv:2211.12701.

Ke, Z., Liu, B., Ma, N., Xu, H., Shu, L., 2021a. Achieving forgetting prevention and knowledge transfer in continual learning. Adv. Neural Inf. Process. Syst. 34, 22443–22456.

Ke, Z., Xu, H., Liu, B., 2021b. Adapting bert for continual learning of a sequence of aspect sentiment classification tasks. arXiv preprint arXiv:2112.03271.

Kendall, A., Gal, Y., Cipolla, R., 2018. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. CVPR.

Kim, B., Doshi-Velez, F., 2021. Machine learning techniques for accountability. AI Mag. 42 (1), 47–52.

Kirstein, F., Wahle, J.P., Ruas, T., Gipp, B., 2022. Analyzing multi-task learning for abstractive text summarization. arXiv preprint arXiv:2210.14606.

Lakshmanan, V., Robinson, S., Munn, M., 2020. Machine Learning Design Patterns. O'Reilly Media.

Lavin, A., Gilligan-Lee, C.M., Visnjic, A., Ganju, S., Newman, D., Ganguly, S., Lange, D., Baydin, A.G., Sharma, A., Gibson, A., et al., 2022. Technology readiness levels for machine learning systems. Nature Commun. 13 (1), 6039.

Lee, H.B., Yang, E., Hwang, S.J., 2018. Deep asymmetric multi-task feature learning. In: Dy, J., Krause, A. (Eds.), Proceedings of the 35th International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 80, PMLR, pp. 2956–2964, URL: https://proceedings.mlr.press/v80/lee18d.html.

Lenyk, Z., Park, J., 2021. Microsoft vision model ResNet-50 combines web-scale data and multi-task learning to achieve state of the art. https://www.microsoft.com/en-us/research/blog/microsoft-vision-model-resnet-50-combines-web-scale-data-and-multi-task-learning-to-achieve-state-of-the-art/. (Accessed 26 March 2024).

Lester, B., Al-Rfou, R., Constant, N., 2021. The power of scale for parameter-efficient prompt tuning. CoRR abs/2104.08691. URL: https://arxiv.org/abs/2104.08691.

Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., Zettlemoyer, L., 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, pp. 7871–7880. http://dx.doi.org/10.18653/v1/2020.acl-main.703, Online. URL: https://aclanthology.org/2020.acl-main.703.

Li, X.L., Liang, P., 2021. Prefix-tuning: Optimizing continuous prompts for generation. arXiv preprint arXiv:2101.00190.

Lin, X., Zhen, H.L., Li, Z., Zhang, Q.F., Kwong, S., 2019. Pareto multi-task learning. In: Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché Buc, F., Fox, E., Garnett, R. (Eds.), In: Advances in Neural Information Processing Systems, vol. 32, Curran Associates, Inc., URL: https://proceedings.neurips.cc/paper/2019/file/685bfde03eb646c27ed565881917c71c-Paper.pdf.

Liu, X., He, P., Chen, W., Gao, J., 2019a. Multi-task deep neural networks for natural language understanding. http://dx.doi.org/10.48550/ARXIV.1901.11504, URL: https://arxiv.org/abs/1901.11504.

Liu, S., Liang, Y., Gitter, A., 2019b. Loss-balanced task weighting to reduce negative transfer in multi-task learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, No. 01. pp. 9977–9978. http://dx.doi.org/10.1609/aaai.v33i01.33019977, URL: https://ojs.aaai.org/index.php/AAAI/article/view/5125.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019c. RoBERTa: A robustly optimized BERT pretraining approach. CoRR abs/1907.11692. URL: http://arxiv.org/abs/1907.11692.

Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., Raffel, C.A., 2022. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. Adv. Neural Inf. Process. Syst. 35, 1950–1965.

Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G., 2021a. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. CoRR abs/2107.13586. URL: https://arxiv.org/abs/2107.13586.

Liu, X., Zheng, Y., Du, Z., Ding, M., Qian, Y., Yang, Z., Tang, J., 2021b. GPT understands, too. CoRR abs/2103.10385. URL: https://arxiv.org/abs/2103.10385.

Long, M., Cao, Z., Wang, J., Yu, P.S., 2017. Learning multiple tasks with multilinear relationship networks. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), In: Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., URL: https://proceedings.neurips.cc/paper/2017/file/03e0704b5690a2dee1861dc3ad3316c9-Paper.pdf.

Lopez-Paz, D., Ranzato, M.A., 2017. Gradient episodic memory for continual learning. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), In: Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., URL: https://proceedings.neurips.cc/paper/2017/file/f87522788a2be2d171666752f97ddebb-Paper.pdf.

Mahabadi, R.K., Ruder, S., Dehghani, M., Henderson, J., 2021. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks. http://dx.doi.org/10.48550/ARXIV.2106.04489, URL: https://arxiv.org/abs/2106.04489.

Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., Bossan, B., 2022. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft.

Marchant, G.E., 2011. The Growing Gap Between Emerging Technologies and the Law. Springer.

McCann, B., Keskar, N.S., Xiong, C., Socher, R., 2018. The natural language decathlon: Multitask learning as question answering. arXiv preprint arXiv:1806.08730.

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., Galstyan, A., 2021. A survey on bias and fairness in machine learning. ACM Comput. Surv. 54 (6), 1–35.

Nahar, N., Zhang, H., Lewis, G., Zhou, S., Kästner, C., 2023. A meta-summary of challenges in building products with ml components–collecting experiences from 4758+ practitioners. In: 2023 IEEE/ACM 2nd International Conference on AI Engineering–Software Engineering for AI. CAIN, IEEE, pp. 171–183.

Nahar, N., Zhou, S., Lewis, G., Kästner, C., 2022. Collaboration challenges in building ml-enabled systems: Communication, documentation, engineering, and process. In: Proceedings of the 44th International Conference on Software Engineering. pp. 413–425.

Pacheco, F., Exposito, E., Gineste, M., Baudoin, C., Aguilar, J., 2018. Towards the deployment of machine learning solutions in network traffic classification: A systematic survey. IEEE Commun. Surv. Tutor. 21 (2), 1988–2014.

Paleyes, A., Urma, R.G., Lawrence, N.D., 2022. Challenges in deploying machine learning: a survey of case studies. ACM Comput. Surv. 55 (6), 1–29.

Palma, R., Martí, L., Sánchez-Pi, N., 2021. Predicting mining industry accidents with a multitask learning approach. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, No. 17. pp. 15370–15376.

Parmar, M., Mishra, S., Purohit, M., Luo, M., Mohammad, M., Baral, C., 2022. In-boXBART: Get instructions into biomedical multi-task learning. In: Findings of the Association for Computational Linguistics: NAACL 2022. Association for Computational Linguistics, Seattle, United States, pp. 112–128. http://dx.doi.org/10.18653/v1/2022.findings-naacl.10, URL: https://aclanthology.org/2022.findings-naacl.10.

Pascal, L., Michiardi, P., Bost, X., Huet, B., Zuluaga, M.A., 2021. Maximum roaming multi-task learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, No. 10. pp. 9331–9341. http://dx.doi.org/10.1609/aaai.v35i10.17125, URL: https://ojs.aaai.org/index.php/AAAI/article/view/17125.

Pei, Z., Liu, L., Wang, C., Wang, J., 2022. Requirements engineering for machine learning: A review and reflection. In: 2022 IEEE 30th International Requirements Engineering Conference Workshops. REW, IEEE, pp. 166–175.

Perera, V., Chung, T., Kollar, T., Strubell, E., 2018. Multi-task learning for parsing the alexa meaning representation language. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 32, No. 1.

Petroni, F., Rocktäschel, T., Riedel, S., Lewis, P., Bakhtin, A., Wu, Y., Miller, A., 2019. Language models as knowledge bases? In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. EMNLP-IJCNLP, Association for Computational Linguistics, Hong Kong, China, pp. 2463–2473. http://dx.doi.org/10.18653/v1/D19-1250, URL: https://aclanthology.org/D19-1250.

Pfeiffer, J., Kamath, A., Rücklé, A., Cho, K., Gurevych, I., 2020a. AdapterFusion: Non-destructive task composition for transfer learning. arXiv preprint arXiv:2005.00247.

Pfeiffer, J., Rücklé, A., Poth, C., Kamath, A., Vulić, I., Ruder, S., Cho, K., Gurevych, I., 2020b. Adapterhub: A framework for adapting transformers. arXiv preprint arXiv:2007.07779.

Pfeiffer, J., Ruder, S., Vulić, I., Ponti, E.M., 2023. Modular deep learning. http://dx.doi.org/10.48550/ARXIV.2302.11529, URL: https://arxiv.org/abs/2302.11529.

Pfeiffer, J., Vulic, I., Gurevych, I., Ruder, S., 2020c. MAD-X: An adapter-based framework for multi-task cross-lingual transfer. CoRR abs/2005.00052. URL: https://arxiv.org/abs/2005.00052.

Pilault, J., Elhattami, A., Pal, C., 2020. Conditionally adaptive multi-task learning: Improving transfer learning in NLP using fewer parameters & less data. http://dx.doi.org/10.48550/ARXIV.2009.09139, URL: https://arxiv.org/abs/2009.09139.

Politou, E., Alepis, E., Patsakis, C., 2018. Forgetting personal data and revoking consent under the GDPR: Challenges and proposed solutions. J. Cybersecur. 4 (1), tyy001.

Polyzotis, N., Roy, S., Whang, S.E., Zinkevich, M., 2018. Data lifecycle challenges in production machine learning: a survey. ACM SIGMOD Rec. 47 (2), 17–28.

Pramanik, S., Agrawal, P., Hussain, A., 2019. OmniNet: A unified architecture for multi-modal multi-task learning. CoRR abs/1907.07804. URL: http://arxiv.org/abs/1907.07804.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al., 2019. Language models are unsupervised multitask learners. OpenAI Blog 1 (8), 9.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J., et al., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. J. Mach. Learn. Res. 21 (140), 1–67.

Rebuffi, S.A., Bilen, H., Vedaldi, A., 2017. Learning multiple visual domains with residual adapters. Adv. Neural Inf. Process. Syst. 30.

Ren, K., Zheng, T., Qin, Z., Liu, X., 2020. Adversarial attacks and defenses in deep learning. Engineering 6 (3), 346–360.

Renggli, C., Karlaš, B., Ding, B., Liu, F., Schawinski, K., Wu, W., Zhang, C., 2019. Continuous integration of machine learning models with ease. ml/ci: Towards a rigorous yet practical treatment. Proc. Mach. Learn. Syst. 1, 322–333.

Rosenberg, I., Shabtai, A., Elovici, Y., Rokach, L., 2021. Adversarial machine learning attacks and defense methods in the cyber security domain. ACM Comput. Surv. 54 (5), 1–36.

Rücklé, A., Geigle, G., Glockner, M., Beck, T., Pfeiffer, J., Reimers, N., Gurevych, I., 2020. Adapterdrop: On the efficiency of adapters in transformers. arXiv preprint arXiv:2010.11918.

Ruder, S., 2017. An overview of multi-task learning in deep neural networks. arXiv preprint arXiv:1706.05098.

Ruder, S., Bingel, J., Augenstein, I., Søgaard, A., 2017. Sluice networks: Learning what to share between loosely related tasks. 2, arXiv preprint arXiv:1705.08142.

Salmani, M., Ghafouri, S., Sanaee, A., Razavi, K., Mühlhäuser, M., Doyle, J., Jamshidi, P., Sharifi, M., 2023. Reconciling high accuracy, cost-efficiency, and low latency of inference serving systems. In: Proceedings of the 3rd Workshop on Machine Learning and Systems. pp. 78–86.

Samant, R.M., Bachute, M., Gite, S., Kotecha, K., 2022. Framework for deep learning-based language models using multi-task learning in natural language understanding: A systematic literature review and future directions. IEEE Access.

Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., Aroyo, L.M., 2021. "Everyone wants to do the model work, not the data work": Data cascades in high-stakes AI. In: Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems. pp. 1–15.

Sanh, V., Webson, A., Raffel, C., Bach, S.H., Sutawika, L., Alyafeai, Z., Chaffin, A., Stiegler, A., Scao, T.L., Raja, A., Dey, M., Bari, M.S., Xu, C., Thakker, U., Sharma, S.S., Szczechla, E., Kim, T., Chhablani, G., Nayak, N., Datta, D., Chang, J., Jiang, M.T.J., Wang, H., Manica, M., Shen, S., Yong, Z.X., Pandey, H., Bawden, R., Wang, T., Neeraj, T., Rozen, J., Sharma, A., Santilli, A., Fevry, T., Fries, J.A., Teehan, R., Bers, T., Biderman, S., Gao, L., Wolf, T., Rush, A.M., 2021. Multitask prompted training enables zero-shot task generalization. http://dx.doi.org/10.48550/ARXIV.2110.08207, URL: https://arxiv.org/abs/2110.08207.

Schröder, T., Schulz, M., 2022. Monitoring machine learning models: a categorization of challenges and methods. Data Sci. Manage. 5 (3), 105–116.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.F., Dennison, D., 2015. Hidden technical debt in machine learning systems. Adv. Neural Inf. Process. Syst. 28.

Serra, J., Suris, D., Miron, M., Karatzoglou, A., 2018. Overcoming catastrophic forgetting with hard attention to the task. In: Dy, J., Krause, A. (Eds.), Proceedings of the 35th International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 80, PMLR, pp. 4548–4557, URL: https://proceedings.mlr.press/v80/serra18a.html.

Sharir, O., Peleg, B., Shoham, Y., 2020. The cost of training nlp models: A concise overview. arXiv preprint arXiv:2004.08900.

Shin, T., Razeghi, Y., Logan IV, R.L., Wallace, E., Singh, S., 2020. AutoPrompt: Eliciting knowledge from language models with automatically generated prompts. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. EMNLP, Association for Computational Linguistics, pp. 4222–4235. http://dx.doi.org/10.18653/v1/2020.emnlp-main.346, Online. URL: https://aclanthology.org/2020.emnlp-main.346.

Sinha, A., Chen, Z., Badrinarayanan, V., Rabinovich, A., 2018. Gradient adversarial training of neural networks. CoRR abs/1806.08028. URL: http://arxiv.org/abs/1806.08028.

Standley, T., Zamir, A., Chen, D., Guibas, L., Malik, J., Savarese, S., 2020. Which tasks should be learned together in multi-task learning? In: III, H.D., Singh, A. (Eds.), Proceedings of the 37th International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 119, PMLR, pp. 9120–9132, URL: https://proceedings.mlr.press/v119/standley20a.html.

Stickland, A.C., Murray, I., 2019. Bert and pals: Projected attention layers for efficient adaptation in multi-task learning. In: International Conference on Machine Learning. PMLR, pp. 5986–5995.

Strubell, E., Ganesh, A., McCallum, A., 2020. Energy and policy considerations for modern deep learning research. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34. pp. 13693–13696.

Sun, R.Y., 2020. Optimization for deep learning: An overview. J. Oper. Res. Soc. China 8 (2), 249–294.

Sun, Y., Wang, S., Li, Y., Feng, S., Tian, H., Wu, H., Wang, H., 2020. Ernie 2.0: A continual pre-training framework for language understanding. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34. pp. 8968–8975.

Takeuchi, H., Yamamoto, S., 2020. Business analysis method for constructing business–AI alignment model. Procedia Comput. Sci. 176, 1312–1321.

Tay, Y., Zhao, Z., Bahri, D., Metzler, D., Juan, D., 2020. HyperGrid: Efficient multi-task transformers with grid-wise decomposable hyper projections. CoRR abs/2007.05891. URL: https://arxiv.org/abs/2007.05891.

Thung, K.H., Wee, C.Y., 2018. A brief review on multi-task learning. Multimedia Tools Appl. 77 (22), 29705–29725.

Upadhyay, R., Phlypo, R., Saini, R., Liwicki, M., 2021. Sharing to learn and learning to share-fitting together meta-learning, multi-task learning, and transfer learning: A meta review. arXiv preprint arXiv:2111.12146.

Üstün, A., Bisazza, A., Bouma, G., van Noord, G., Ruder, S., 2022. Hyper-X: A unified hypernetwork for multi-task multilingual transfer. arXiv preprint arXiv:2205.12148.

Vafaeikia, P., Namdar, K., Khalvati, F., 2020. A brief review of deep multi-task learning and auxiliary task learning. arXiv preprint arXiv:2007.01126.

Vandenhende, S., Georgoulis, S., Van Gansbeke, W., Proesmans, M., Dai, D., Van Gool, L., 2021. Multi-task learning for dense prediction tasks: A survey. IEEE Trans. Pattern Anal. Mach. Intell..

Vartak, M., Madden, S., 2018. Modeldb: Opportunities and challenges in managing machine learning models. IEEE Data Eng. Bull. 41 (4), 16–25.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.

Von Oswald, J., Henning, C., Sacramento, J., Grewe, B.F., 2019. Continual learning with hypernetworks. arXiv preprint arXiv:1906.00695.

Vu, T., Wang, T., Munkhdalai, T., Sordoni, A., Trischler, A., Mattarella-Micke, A., Maji, S., Iyyer, M., 2020. Exploring and predicting transferability across NLP tasks. CoRR abs/2005.00770. URL: https://arxiv.org/abs/2005.00770.

Wang, S., Khabsa, M., Ma, H., 2020. To pretrain or not to pretrain: Examining the benefits of pretraining on resource rich tasks. arXiv preprint arXiv:2006.08671.

Wang, L., Li, R., Yan, Y., Yan, Y., Wang, S., Wu, W., Xu, W., 2022. InstructionNER: A multi-task instruction-based generative framework for few-shot NER. http://dx.doi.org/10.48550/ARXIV.2203.03903, URL: https://arxiv.org/abs/2203.03903.

Wang, A., Pruksachatkun, Y., Nangia, N., Singh, A., Michael, J., Hill, F., Levy, O., Bowman, S., 2019. Superglue: A stickier benchmark for general-purpose language understanding systems. Adv. Neural Inf. Process. Syst. 32.

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., Bowman, S.R., 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. arXiv preprint arXiv:1804.07461.

Whang, S.E., Roh, Y., Song, H., Lee, J.G., 2023. Data collection and quality challenges in deep learning: A data-centric ai perspective. VLDB J. 1–23.

Worsham, J., Kalita, J., 2020. Multi-task learning for natural language processing in the 2020s: where are we going? Pattern Recognit. Lett. 136, 120–126.

Wu, N., Xie, Y., 2022. A survey of machine learning for computer architecture and systems. ACM Comput. Surv. 55 (3), 1–39.

Wu, S., Zhang, H.R., Ré, C., 2020. Understanding and improving information transfer in multi-task learning. http://dx.doi.org/10.48550/ARXIV.2005.00944, URL: https://arxiv.org/abs/2005.00944.

Yang, C., Brower-Sinning, R., Lewis, G.A., Kästner, C., Wu, T., 2022. Capabilities for better ML engineering. arXiv preprint arXiv:2211.06409.

Yang, Y., Hospedales, T.M., 2016. Trace norm regularised deep multi-task learning. CoRR abs/1606.04038. URL: http://arxiv.org/abs/1606.04038.

Yu, Y., Yang, C.H.H., Kolehmainen, J., Shivakumar, P.G., Gu, Y., Ren, S.R.R., Luo, Q., Gourav, A., Chen, I.F., Liu, Y.C., et al., 2023. Low-rank adaptation of large language model rescoring for parameter-efficient speech recognition. In: 2023 IEEE Automatic Speech Recognition and Understanding Workshop. ASRU, IEEE, pp. 1–8.

Zamir, A.R., Sax, A., Shen, W., Guibas, L.J., Malik, J., Savarese, S., 2018. Taskonomy: Disentangling task transfer learning. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. CVPR.

Zhai, A., Wu, H.Y., Tzeng, E., Park, D.H., Rosenberg, C., 2019. Learning a unified embedding for visual search at pinterest. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 2412–2420.

Zhang, Q., Chen, M., Bukharin, A., He, P., Cheng, Y., Chen, W., Zhao, T., 2023a. Adaptive budget allocation for parameter-efficient fine-tuning. arXiv preprint arXiv:2303.10512.

Zhang, L., Shi, J., Wang, L., Xu, C., 2020. Electricity, heat, and gas load forecasting based on deep multitask learning in industrial-park integrated energy system. Entropy 22 (12), http://dx.doi.org/10.3390/e22121355, URL: https://www.mdpi.com/1099-4300/22/12/1355.

Zhang, Y., Yang, Q., 2017. A survey on multi-task learning. arXiv preprint arXiv:1707.08114.

Zhang, Y., Yang, Q., 2018. An overview of multi-task learning. Natl. Sci. Rev. 5 (1), 30–43.

Zhang, Z., Yu, W., Yu, M., Guo, Z., Jiang, M., 2023b. A survey of multi-task learning in natural language processing: Regarding task relatedness and training methods. In: Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics. Association for Computational Linguistics, Dubrovnik, Croatia, pp. 943–956, URL: https://aclanthology.org/2023.eacl-main.66.

Zheng, F., Deng, C., Sun, X., Jiang, X., Guo, X., Yu, Z., Huang, F., Ji, R., 2019. Pyramidal person re-identification via multi-loss dynamic training. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR.

Zhou, W., 2019. An overview of models and methods for multi-task learning. [CSCI-699 - advanced topics in representation learning for NLP @ USC]. URL: https://shanzhenren.github.io/csci-699-replnlp-2019fall/lectures/W6-L1-Multi_Task_Learning.pdf.